

Python Backend Flow Guide

Research Notebook backend: endpoint flows, internal services, and data movement

This guide explains what happens when each backend endpoint is hit. The Python backend uses FastAPI routes, Keycloak authentication, SQLAlchemy repositories, Qdrant vector search, and Gemini for embeddings and answers.

Big Picture Flow

```
Frontend request
  |
  v
FastAPI route in src/main.py
  |
  v
Keycloak auth dependency
  |
  v
Repository or service layer
  |
  v
Postgres / Qdrant / Gemini as needed
  |
  v
JSON response to frontend
```

Core Components

- `src/main.py`: Defines HTTP endpoints and coordinates dependencies.
- `src/auth.py`: Verifies Keycloak JWT tokens and returns the current user.
- `src/database.py`: Creates the SQLAlchemy engine and per-request database session.
- `src/models.py`: Maps Notebook, Document, and Chunk tables.
- `src/repositories.py`: Contains Postgres operations for notebooks, documents, and chunks.
- `src/services/*`: Contains chunking, indexing, search, chat, and answer generation logic.
- `src/utils/*`: Contains helper logic such as text extraction and chunk splitting.

GET	/protected	Check that Keycloak authentication works
-----	------------	--

Protected Route Flow

```
Request includes Authorization: Bearer <token>
  |
  v
get_current_user() reads the header
  |
  v
PyJWKClient fetches Keycloak public keys
  |
  v
jwt.decode() verifies signature and issuer
  |
  v
Route receives user dict
  |
  v
Backend returns message and user
```

Route code

```
@app.get('/protected')
```

```
def protected_route(user: dict = Depends(get_current_user)):
    return {
        'message': 'Protected route access granted',
        'user': user,
    }
```

Auth code

```
def get_current_user(authorization: str | None = Header(None)) -> dict:
    if not authorization:
        raise HTTPException(status_code=401, detail='Missing Authorization header')

    token = authorization.split()[1]
    signing_key = jwks_client.get_signing_key_from_jwt(token)
    decoded = jwt.decode(token, signing_key.key, algorithms=['RS256'],
        issuer=settings.keycloak_issuer)

    return {
        'id': decoded.get('sub'),
        'username': decoded.get('preferred_username'),
        'email': decoded.get('email'),
    }
```

POST	/notebooks	Create a notebook for the logged-in user
------	------------	--

Create Notebook Flow

Example request body

```
{ 'title': 'Research Notes' }
```

Request body is validated by NotebookCreate

↓
Keycloak token identifies the user

↓
get_db() opens a database session

↓
NotebookRepository.create() inserts the row

↓
Created notebook is returned

Route code

```
@app.post('/notebooks', response_model=NotebookOut, status_code=201)
def create_notebook(payload: NotebookCreate, db: Session = Depends(get_db), user: dict =
    Depends(get_current_user)):
    return notebook_repo.create(db, title=payload.title, user_id=user['id'])
```

Repository code

```
def create(self, db: Session, *, title: str, user_id: str) -> Notebook:
    notebook = Notebook(title=title, userId=user_id)
    db.add(notebook)
    db.commit()
    db.refresh(notebook)
    return notebook
```

GET	/notebooks	List current user notebooks
-----	------------	-----------------------------

Get Notebooks Flow

Auth verifies user

↓
Repository filters Notebook.userId by token subject
↓

```

      v
Rows are ordered newest first
      |
      v
List is returned

```

Route code

```

@app.get('/notebooks', response_model=list[NotebookOut])
def get_notebooks(db: Session = Depends(get_db), user: dict = Depends(get_current_user)):
    return notebook_repo.find_by_user(db, user['id'])

```

Repository code

```

def find_by_user(self, db: Session, user_id: str) -> list[Notebook]:
    return list(db.scalars(select(Notebook).where(Notebook.userId ==
        user_id).order_by(Notebook.createdAt.desc()))))

```

PUT	/notebooks/{notebook_id}	Rename a notebook owned by the user
------------	---------------------------------	-------------------------------------

Update Notebook Flow

Example request body

```

{ 'title': 'Updated Research Notes' }

```

Auth verifies user

```

      |
      v
Update runs only when id and userId both match
      |
      v
If rowcount is zero return 404
      |
      v
Otherwise return success

```

Route code

```

@app.put('/notebooks/{notebook_id}')
def update_notebook(notebook_id: str, payload: NotebookUpdate, db: Session =
    Depends(get_db), user: dict = Depends(get_current_user)):
    count = notebook_repo.update(db, id=notebook_id, user_id=user['id'],
        title=payload.title)
    if count == 0:
        return JSONResponse(status_code=404, content={'error': 'Notebook not found'})
    return {'success': True}

```

Repository code

```

def update(self, db: Session, *, id: str, user_id: str, title: str) -> int:
    result = db.execute(update(Notebook).where(Notebook.id == id, Notebook.userId ==
        user_id).values(title=title))
    db.commit()
    return result.rowcount or 0

```

DELETE	/notebooks/{notebook_id}	Delete a notebook owned by the user
---------------	---------------------------------	-------------------------------------

Delete Notebook Flow

Auth verifies user

```

      |
      v
Delete runs only for id plus userId
      |
      v
If nothing deleted return 404
      |
      v
Otherwise return success

```

Route code

```
@app.delete('/notebooks/{notebook_id}')
def delete_notebook(notebook_id: str, db: Session = Depends(get_db), user: dict = Depends(get_current_user)):
    count = notebook_repo.delete(db, id=notebook_id, user_id=user['id'])
    if count == 0:
        return JSONResponse(status_code=404, content={'error': 'Notebook not found'})
    return {'success': True}
```

Repository code

```
def delete(self, db: Session, *, id: str, user_id: str) -> int:
    result = db.execute(delete(Notebook).where(Notebook.id == id, Notebook.userId == user_id))
    db.commit()
    return result.rowcount or 0
```

POST	/documents/{notebook_id}	Upload a plain text document
------	--------------------------	------------------------------

Upload Document Flow

```
Frontend uploads multipart file field named file
|
v
Auth verifies user
|
v
extract_text() reads text/plain content
|
v
DocumentRepository.create() stores filename and content
|
v
Saved document is returned
```

```
@app.post('/documents/{notebook_id}', response_model=DocumentOut, status_code=201)
async def upload_document(notebook_id: str, file: UploadFile = File(...), db: Session = Depends(get_db), user: dict = Depends(get_current_user)):
    text = await extract_text(file)
    return document_repo.create(db, notebook_id=notebook_id, user_id=user['id'], filename=file.filename or 'upload.txt', content=text)

async def extract_text(file: UploadFile) -> str:
    if file.content_type == 'text/plain':
        content = await file.read()
        return content.decode('utf-8')
    raise HTTPException(status_code=400, detail='Unsupported file type')
```

GET	/documents/{notebook_id}	List documents in a notebook
-----	--------------------------	------------------------------

Get Documents Flow

```
Auth verifies user
|
v
Repository filters by notebookId and userId
|
v
Rows are ordered newest first
|
v
Documents are returned
```

```
@app.get('/documents/{notebook_id}', response_model=list[DocumentOut])
def get_documents(notebook_id: str, db: Session = Depends(get_db), user: dict = Depends(get_current_user)):
    return document_repo.find_by_notebook(db, notebook_id=notebook_id, user_id=user['id'])
```

Chunk Document Flow

```

Auth verifies user
  |
  v
Find document by documentId and userId
  |
  v
Delete old chunks for safe reruns
  |
  v
Split text into overlapping chunks
  |
  v
Save chunks in Postgres
  |
  v
Embed chunk text with Gemini
  |
  v
Upsert vectors into Qdrant
  |
  v
Return chunksCreated

@app.post('/chunks/{document_id}')
def chunk_document_route(document_id: str, db: Session = Depends(get_db), user: dict = Depends(get_current_user)):
    document = document_repo.find_by_id(db, id=document_id, user_id=user['id'])
    if not document:
        return JSONResponse(status_code=404, content={'error': 'Document not found'})
    result = chunk_document(db, document=document, user_id=user['id'])
    return {'message': 'Document chunked', 'chunksCreated': result['count']}

def chunk_document(db: Session, *, document: Document, user_id: str) -> dict:
    repo.delete_by_document(db, document_id=document.id, user_id=user_id)
    chunks = chunk_text(document.content)
    repo.bulk_create(db, document_id=document.id, user_id=user_id, chunks=chunks)
    stored_chunks = repo.find_by_document(db, document_id=document.id, user_id=user_id)
    index_chunks(chunks=stored_chunks)
    return {'count': len(stored_chunks)}

def chunk_text(text: str) -> list[dict]:
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100,
        separators=['\n\n', '\n', ' ', ''])
    docs = splitter.create_documents([text])
    return [{ 'content': doc.page_content, 'order': index} for index, doc in enumerate(docs)]

```

Indexing Internals

```

ensure_collection() creates Qdrant collection if missing
  |
  v
Gemini embedding model converts each chunk to a vector
  |
  v
Point id is the chunk id
  |
  v
Payload stores documentId, userId, and order
  |
  v
Qdrant stores the searchable vectors

def index_chunks(*, chunks: list[Chunk]) -> None:
    ensure_collection()
    vectors = get_embeddings().embed_documents([chunk.content for chunk in chunks])
    points = [PointStruct(id=chunk.id, vector=vectors[index], payload={'documentId':
        chunk.documentId, 'userId': chunk.userId, 'order': chunk.order}) for index, chunk in
        enumerate(chunks)]
    get_qdrant().upsert(collection_name=settings.qdrant_collection, points=points)

```

POST	/search	Semantic search over a document
------	---------	---------------------------------

Search Flow

Example request body

```
{ 'documentId': 'document-id', 'query': 'What is the methodology?' }
```

Auth verifies user

|

v

Query text is embedded with Gemini

|

v

Qdrant compares query vector to chunk vectors

|

v

Filter restricts to documentId and userId

|

v

Top matching chunks are returned

```
def search_chunks(*, document_id: str, user_id: str, query: str, limit: int = 5):
    vector = get_embeddings().embed_query(query)
    return get_qdrant().search(
        collection_name=settings.qdrant_collection,
        query_vector=vector,
        limit=limit,
        query_filter=Filter(must=[
            FieldCondition(key='documentId', match=MatchValue(value=document_id)),
            FieldCondition(key='userId', match=MatchValue(value=user_id)),
        ]),
    )
```

POST	/chat	Ask a question about a document
------	-------	---------------------------------

Chat Flow

Example request body

```
{ 'documentId': 'document-id', 'question': 'Summarize the main idea' }
```

Auth verifies user

|

v

Question is embedded

|

v

Qdrant finds relevant chunk ids

|

v

Postgres fetches full chunk text

|

v

Prompt is built from chunks plus question

|

v

Gemini writes answer

|

v

Backend returns answer and source chunk ids

```
def chat_with_document(db: Session, *, document_id: str, user_id: str, question: str) -> dict:
    results = search_chunks(document_id=document_id, user_id=user_id, query=question,
                             limit=5)
    chunk_ids = [str(result.id) for result in results]
    chunks = chunk_repo.find_by_ids(db, ids=chunk_ids, user_id=user_id)
    answer = answer_question(question=question, chunks=chunks)
    return {'answer': answer, 'sources': chunk_ids}
```

```
def answer_question(*, question: str, chunks: list[Chunk]) -> str:
    context = '\n\n'.join(f'Chunk {index + 1}: \n{chunk.content}' for index, chunk in
        enumerate(chunks))
    prompt = f'''You are a helpful assistant.
    Answer the question using ONLY the context below.
    If the answer is not in the context, say you don't know.

    Context:
    {context}

    Question:
    {question}

    Answer:'''
    response = get_llm().invoke(prompt)
    return response.content
```

Full User Journey

- User logs in through Keycloak and frontend receives an access token.
- Frontend creates a notebook with POST /notebooks.
- Frontend uploads a .txt document with POST /documents/{notebook_id}.
- Backend stores the document content in Postgres.
- Frontend triggers POST /chunks/{document_id}.
- Backend splits the document into chunks and stores them in Postgres.
- Backend embeds chunks with Gemini and stores vectors in Qdrant.
- User asks a question with POST /chat.
- Backend embeds the question and searches Qdrant.
- Backend fetches real chunk text from Postgres.
- Gemini answers using only the retrieved context.
- Frontend displays the answer and source chunk ids.

Mental Model

```
Postgres = real app data: notebooks, documents, chunks
Qdrant   = vector search over chunks
Gemini   = embeddings plus final answer generation
```

```
For chat:
Postgres gives actual text
Qdrant finds relevant text
Gemini writes the answer
```